

FORMATION PROFESSIONNELLE • 2 JOURS

Formation OpenAI Codex

Coder avec l'IA, tout en gardant la maîtrise du code

Jour 1 • Prise en main

Jour 2 • Tools & Agents



```
> Ajoute des tests au module  
facture
```

- ✓ Analyse
- ✓ Plan
- ✓ Code + tests
- ▶ Relecture humaine

Votre formateur



Daouda DIOP

Senior Data Scientist & Expert IA Générative



Doctorat en Physique – Optique & Computer Vision



Chef de projet R&D – Ingénieur R&D Optique/Computer Vision



Senior Data Analyst / Data Scientist



Formateur certifié – LLM, RAG, Agents IA

PARCOURS • DANS L'ORDRE

1



Doctorat en Physique
Optique / Computer
Vision

2



Chef de projet
fibre optique

3



Ingénieur R&D
Optique / Computer
Vision

4



Ingénieur R&D Couches
minces
Ingénieur R&D Optique /
Computer Vision

5



Chef de projet R&D
Optique / Computer
Vision · Senior Data
Analyst / Data Scientist

Depuis 2019

En parallèle



Chef d'entreprise
(auto-entrepreneur)

Tour de table : qui êtes-vous ?



Votre rôle et votre contexte

Métier, langages, environnement de développement.



Votre expérience de l'IA pour coder

Déjà utilisateur régulier, débutant, ou curieux ?



Vos attentes

Ce que vous voulez savoir faire à la fin des 2 jours.



Un cas d'usage concret

Une tâche de votre entreprise à traiter avec Codex.



1 minute chacun

Vos réponses me permettent
d'adapter les exemples et les
démonstrations à vos besoins réels.

Ce que vous saurez faire dans 2 jours



Prompter

Structurer un prompt efficace pour générer du code fiable.



Configurer

Configurer Codex dans son IDE et gérer un projet multi-fichiers.



Actions

Utiliser les actions de Codex pour documenter, tester et refactorer.



Tools

Créer des Tools personnalisés avec function calling.



Agents

Mettre en place des agents spécialisés pour automatiser le développement.



Sécuriser

Sécuriser l'exécution du code et valider les résultats.

À qui s'adresse cette formation ?



Public visé

Développeur·se·s souhaitant accélérer leur productivité avec Codex tout en gardant la maîtrise du code.

Pré-requis



Connaître les bases d'un langage de programmation

Python, JS, Java...



Savoir utiliser un IDE

VS Code, JetBrains...



Avoir un abonnement payant actif

Solo ou entreprise



Apporter un ordinateur portable

Le programme en un coup d'œil

JOUR 1

Prise en main de Codex & génération assistée

- 1 Introduction à OpenAI Codex
- 2 Comprendre le prompting pour le code
- 3 Installation & configuration
- 4 Codex en interaction
- 5 Les Actions
- 6 Mises en pratique

JOUR 2

Fonctions avancées, Tools & Agents spécialisés

- 1 Actions avancées & customisation
- 2 Les Tools OpenAI
- 3 Agents & orchestration multi-agents
- 4 Exécution sécurisée du code
- 5 Hooks & commandes personnalisées
- 6 Mises en pratique

Organisation d'une journée

Déroulement type



Comprendre

J'explique les concepts avec des mots simples.



Voir

Je fais une démonstration en direct.



Pratiquer

Vous mettez en œuvre sur votre poste.

Un projet fil rouge sur les 2 jours



Génération de code

Structure du projet, modules,
tests



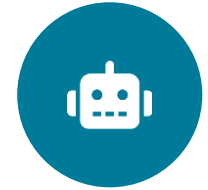
Automatisation

Actions et tools
personnalisés



Refactor assisté

Améliorer l'existant en
gardant le contrôle



Agents dédiés

Des agents spécialisés qui
collaborent

Au fil de la formation, vous développerez un mini-projet en combinant génération de code, automatisation, refactor assisté et agents dédiés.

01

JOUR 1

Prise en main de Codex & génération assistée

- 1 Introduction à OpenAI Codex
- 2 Comprendre le prompting pour le code
- 3 Installation & configuration
- 4 Codex en interaction
- 5 Les Actions
- 6 Mises en pratique

Qu'est-ce qu'OpenAI Codex ?

L'outil d'OpenAI dédié au développement logiciel

Un assistant de programmation basé sur l'IA, qui travaille avec vous sur votre code.

Ce qu'on peut lui demander

- Explorer et comprendre du code
- Construire une nouvelle fonctionnalité
- Relire du code et proposer des changements
- Corriger des problèmes et des échecs

DISPONIBLE SOUS PLUSIEURS FORMES



Application de bureau



Version web



Ligne de commande (CLI)



Extension pour éditeurs de code



Environnement cloud

Source : documentation officielle OpenAI. La liste évolue : vérifiez la version que vous utilisez.

Codex face aux autres assistants de code

OpenAI Codex

GitHub Copilot

Cursor

Claude Code

Six critères pour comparer, quel que soit l'outil



Où il s'exécute

Éditeur, terminal, cloud ?



Autonomie

Il propose, ou il agit ?



Modèles

Lesquels sont disponibles ?



Contexte

Comment lui donner le contexte du projet ?



Sécurité

Confidentialité et garanties ?



Coût

Licences et facturation ?

Cas d'usage : à quoi ça sert concrètement ?



Génération de code

Décrire le besoin, obtenir une première version.



Refactor

Améliorer la structure sans changer le comportement.



Documentation

Commentaires, descriptions, README.



Tests

Proposer des tests pour un module.



Outils internes

Scripts et petites applications pour votre entreprise.

Le vocabulaire essentiel



Prompt

Le message que vous envoyez au modèle : votre demande, en langage naturel.

Analogie : la question posée à un collègue.



Instructions

Les consignes qui définissent le rôle, le style et les règles du modèle.

Analogie : la fiche de poste.



Context window

La quantité maximale de texte que le modèle peut prendre en compte en une fois.

Analogie : la surface du bureau.



Tokens

Les morceaux de texte (mots ou parties de mots) lus et écrits par le modèle. Ils mesurent le contexte et le coût.

Analogie : les pièces d'un puzzle.

Structurer un prompt technique



Objectif

Ce que vous voulez obtenir.



Contraintes

Ce qu'il faut respecter.



Style

Les conventions de code.



Format attendu

La forme du résultat.

EXEMPLE ILLUSTRATIF

Objectif : ajouter une fonction qui calcule la TVA d'une facture.

Contraintes : Python 3.11, aucune dépendance externe, ne pas modifier les fonctions existantes.

Style : PEP 8, noms explicites, docstring courte.

Format attendu : le code de la fonction, puis 3 tests pytest. Sans explication.

Un même besoin, deux prompts

Prompt vague

« Fais-moi un système de facturation. »

Le modèle doit deviner : langage, base de données, fonctionnalités, format...

Prompt structuré

Objectif : fonction qui calcule le total d'une facture avec TVA.

Contraintes : Python, sans dépendance externe.

Format : le code + des tests.

Moins d'hypothèses = résultat plus fiable = moins d'allers-retours

Les règles d'or du prompt pour le code

1

Un objectif clair par demande

Ne pas mélanger plusieurs tâches.

2

Donner le contexte utile

Pas tout le projet : ce qui sert à la tâche.

3

Préciser contraintes et format

Versions, bibliothèques, forme du résultat.

4

Découper les tâches complexes

Plusieurs petites demandes cadrées.

5

Fournir un exemple attendu

Un exemple vaut souvent une longue explication.

6

Relire, tester, itérer

Vérifier avant d'accepter.

Un contexte persistant pour un projet multi-fichiers

SANS contexte persistant

Vous répétez les mêmes consignes à chaque demande : langage, conventions, commande de tests...

AVEC contexte persistant

Ces informations sont écrites une fois : l'assistant démarre déjà informé du projet.



Historique

Les échanges précédents, à gérer pour ne garder que l'utile.



Filesets

Le choix des fichiers mis à disposition du modèle.



Patterns de prompts

Des formulations réutilisables pour les cas récurrents.

Dans Codex, des instructions de projet peuvent être placées dans un fichier AGENTS.md (documentation officielle).

Accéder à OpenAI : API, Playground, intégrations



L'API

Accès programmatique aux modèles, depuis votre code, avec une clé d'accès. Usage mesuré en tokens.



Le Playground

Interface web pour tester des prompts et des modèles, sans écrire de code.



Les intégrations

Les outils qui relient les modèles à votre quotidien, notamment les extensions d'IDE.



Votre clé d'accès est un secret : jamais en clair dans le code, jamais dans un dépôt Git partagé. Utilisez une variable d'environnement ou un gestionnaire de secrets.

Installer Codex dans votre IDE



VS Code



JetBrains



Neovim

LES 4 ÉTAPES

1

Installer l'extension
dans l'éditeur



2

Se connecter avec son
compte



3

Ouvrir son projet



4

Vérifier avec une
première demande
simple

Nous suivrons la procédure officielle du moment, en direct.

Paramétrer les modèles



Qualité

Justesse et pertinence du code produit.



Vitesse

Temps de réponse pour une demande.



Coût

Consommation en tokens facturée à l'usage.

Modèles cités au programme

gpt-4.1

Modèle généraliste

o3-mini

Modèle de raisonnement

o4-mini

Modèle de raisonnement

etc.

La liste évolue

Testez sur vos propres cas d'usage : comparez qualité, temps de réponse et coût.

Le chat de Codex : générer, expliquer, corriger



Générer

« Écris une fonction qui valide une adresse e-mail. »

Exemple de demande



Expliquer

« Explique-moi ce que fait ce module. »

Exemple de demande



Corriger

« Ce test échoue avec cette erreur : ... Corrige le code. »

Exemple de demande

Appel de fichiers et navigation intelligente

PROJET (EXEMPLE)

```
mon-projet/  
  
  src/  
  
    facture.py  
  
    client.py  
  
    paiement.py  
  
  tests/  
  
    test_facture.py  
  
  README.md
```



Analyser et modifier le code existant

Vous désignez les fichiers concernés.



Résumer plusieurs modules

Comprendre comment ils fonctionnent ensemble.

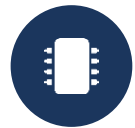


Refactor global

Appliquer une amélioration sur plusieurs fichiers.

Plus l'action est large, plus la relecture du diff est importante.

Premiers paramètres : model, temperature, system prompt



model

Le modèle utilisé. Compromis entre qualité, vitesse et coût.



temperature

Le degré de variabilité des réponses. Pour du code, on privilégie en général des valeurs basses : réponses plus stables.



system prompt

La consigne de fond : le rôle, le style, les règles générales.
Ex : « Tu es un développeur Python expérimenté. »

Selon les modèles, certains paramètres ne sont pas ajustables : nous vérifierons dans l'outil.

Les Actions de Codex



Créer & corriger

Create File

Edit File

Fix Problems



Analyser

Review

Explain

Find Bugs



Documenter

Document

Generate README



Piloter

Plan

Todo

Refactor

Intitulés du programme : le libellé exact peut varier selon la version de l'outil.

Suivi des coûts, logs, historique et diffs



Coûts

Suivre la consommation de tokens.



Logs

Garder la trace de ce qui a été fait.



Historique

Retrouver et reprendre les échanges.



Diffs

Comparer l'avant et l'après.

EXEMPLE DE DIFF (ILLUSTRATIF)

```
def total(prix, qte):  
-     return prix * qte  
+     tva = 0.20  
+     return prix * qte * (1 + tva)
```

```
def afficher(t):  
-     print(t)  
+     print(f"{t:.2f} €")
```

Rouge : supprimé · Vert : ajouté

Mises en pratique • Jour 1

1

Initialiser un projet

Générer la structure complète du projet avec Codex.



2

Appeler un prompt via l'API

Envoyer une demande en ligne de commande avec l'API.



3

Module + tests + documentation

Générer un module, ses tests et une documentation cohérente.



Pensez : relire • tester • itérer

Ce qu'il faut retenir du Jour 1



Codex : l'outil OpenAI pour le dev, à comparer avec une grille de critères



Un bon prompt : objectif · contraintes · style · format attendu



Un contexte persistant évite de tout répéter



Les actions font gagner du temps sur les tâches courantes



Le diff est votre outil de contrôle : on relit avant d'accepter



Quiz flash

1. Quels sont les 4 éléments d'un prompt technique ?
2. À quoi sert le diff ?
3. Que règle la temperature ?